

PART A

(PART A : TO BE REFERRED BY STUDENTS)

Experiment No. 03 (A)

Aim : To Design and Implement two pass Macro Processor

Objective: Develop a program to implement two pass macro processor: **a.** To generate Macro definition Table(MDT)

b. To generate Macro Name table(MNT)

c. To generate Argument List Array(ALA)

d. To generate expanded source code

Outcome: Students are able to design and implement two pass Macro

Processor. **Theory:**

A macro processor is a program that copies a stream of text from one place to another, making a systematic set of replacements as it does so. Macro processors are often embedded in other programs, such as assemblers and compilers. Sometimes they are standalone programs that can be used to process any kind of text.

Macro processors have been used for language expansion (defining new language constructs that can be expressed in terms of existing language components), for systematic text replacements that require decision making, and for text reformatting (e.g. conditional extraction of material from an HTML file).

The following four major tasks are done by macro processor.

1. Recognize macro definition by identifying MACRO and MEND pseudo-ops.
2. Save these definitions which are required for macro expansion process
3. Recognize macro calls by identifying macro name which appears as operation mnemonic.
4. Finally expand macro call and substitute arguments for the dummy arguments.

Databases Used

Pass1 Databases:

1. The input source program
2. The output source deck to be used by pass 2
3. The Macro Definition Table (MDT): this table stores the macro definition.
4. The Macro name Table (MNT): This table stores macro names which are defined.
5. The Macro Definition Table Counter (MDTC), which used to indicate next available entry in MDT.
6. The Macro Name Table Counter (MNTC), which used to indicate next available entry in MNT.
7. Argument list Array (ALA), which stores the index markers for dummy arguments.

Pass 2 Databases:

1. Copy of input source program
2. The output for assembler: the expanded source code
3. The Macro Definition Table (MDT): created by pass1.
4. The Macro name Table(MNT) : This created by pass1
5. The Macro Definition Table Pointer (MDTP), which used to indicate next line used in macro expansion.
6. Argument list Array (ALA), which is used to substitute arguments for index markers.

Format of databases:

We will use following example to discuss the format of all databases.

```
MACRO
&LAB ADDM &ARG1, &ARG2, &ARG3
A 1,&ARG1
A 2,&ARG2
A 3,&ARG3
MEND
.....
LOOP ADDM D1, D2, D3
.....
```

1. Argument List Array (ALA)

This is an array which stores all arguments used in macro definition. Each argument is assigned an index marker.

Consider following macro call,

LOOP ADDM D1, D2, D3

The ALA for this would be:

	8 bytes per entry
Index	Argument
0	“Loopbbbb”
1	“D1bbbbbb”
2	“D2bbbbbb”
3	“D3bbbbbb”

2. Macro Definition Table (MDT)

This table stores each line of macro definition in it except the line for MACRO pseudo op. The MDT for above example is :

	80 bytes per entry
Index	Card
.....	
10	&LAB ADDM &ARG1, &ARG2, &ARG3
11	A 1,&ARG1

12	A 2,&ARG2
13	A 3,&ARG3
14	MEND
.....	

3. Macro Name Table (MNT)

Each entry in MNT has following fields:

Index:

Name: It is a macro name

MDT Index: This is an index from MDT which indicates the line number from which macro definition is stored in MDT.

The MNT for above example is :

	8 bytes per entry	
index	Name	MDT index
.....		
3	“ADDMbbbb”	10
.....		

Algorithm

Pass 1 processing:

1. Initialize MDTC and MNTC as MDTC=MNTC=1
2. Read a line from input source card.
3. If it is MACRO pseudo-op then
 - 3.1 Read the next line which will be the macro name line. Enter the macro name in MNT with current value of MDTC.
 - 3.2 Increment the value of MNTC by 1
 - 3.3 Then the argument list array is prepared for the arguments found in the macro name line.
 - 3.4 The macro name card is also inserted in MDT.
 - 3.5 Increment the value of MDTC by 1
 - 3.6 Read the next line from source card
 - 3.7 Substitute the index markers for arguments from ALA prepared in previous step and then enter this line into MDT
 - 3.8 Increment the value of MDTC by 1
 - 3.9 Check whether it is MEND pseudo-op then

- 3.9.1 Go to step 2.
- else
- 3.9.2 Go to step 3.6
- 4. Else, if it is not MACRO pseudo-op then simply write the line in the copy of source deck prepared for pass2.
- 5. Check if it is END pseudo-op then
 - 5.1 Go to step 6
 - else
 - 5.2 Go to step 2.
- 6. Stop

Pass 2 processing:

- 1. Read a line from copy of source deck prepared by pass1.
 - 2. Search the MNT for match with the op-code.
 - 3. If macro name found in MNT then
 - 3.1 Initialize MDTP with the value of MDT index from the MNT entry. Thus MDTP holds the location from which macro definition is stored in MDT.
 - 3.2 Argument list array is prepared for the arguments found in macro call.
 - 3.3 Increment value of MDTP by 1.
 - 3.4 Read the next line from MDT and substitute the arguments from macro call for the dummy arguments.
 - 3.5 If it is MEND pseudo-op then
 - 3.5.1 Go to step 1.
 - else
 - 3.5.2 Write this line into expanded source code.
 - 3.5.3 Increment the value of MDTP by 1
 - 3.5.4 Go to step 3.4 in order to process remaining statements from MDT.
 - 4. Else if match for macro name is not found in MNT then write the line into expanded source code.
 - 5. If it is END pseudo-op then
 - 1.1 Supply this expanded source code copy to assembler.
 - 1.2 Go to step 6
 - else
 - 5.3 Go to step 1.
7. STOP

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded at the end of the practical)

Roll. No. C-54	Name: Quazi Md Moinuddin
Class: TE-C	Batch: C-3
Date of Experiment:12/02/25	Date of Submission:19/03/25
Grade:	

B.1 Software Code written by student:

(Paste your code completed during the 2 hours of practical in the lab here)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_MACROS 10
#define MAX_MDT 100
#define MAX_ALA 10
#define MAX_CODE 100

// Structures for tables
typedef struct {
    char name[20]; // Macro name
    int mdt_index; // MDT index where macro starts
} MNTEntry;

typedef struct {
    char definition[50]; // Macro definition line
} MDTEEntry;

typedef struct {
    char formal[20]; // Formal parameter name
    char positional[5]; // Positional argument (e.g., #1, #2)
} ALAEntry;

MNTEntry MNT[MAX_MACROS];
MDTEEntry MDT[MAX_MDT];
ALAEntry ALA[MAX_ALA];

int mnt_count = 0, mdt_count = 0, ala_count = 0;

void processPass1(char *filename);
void processPass2(char *filename);
void displayTables();
void replaceArgumentsWithALA(char *line);
void expandMacro(char *macroName, char *actualParams[]);

int main() {
    char filename[] = "macro_input.txt";

    processPass1(filename);
    displayTables();
    processPass2(filename);

    return 0;
}
```

```

// ----- Pass 1: Macro Definition Processing -----
---- void processPass1(char *filename) {
    FILE *fp = fopen(filename, "r");
    if (!fp) {
        printf("Error opening file!\n");
        return;
    }

    char line[100];
    int inMacro = 0;

    while (fgets(line, sizeof(line), fp)) {
        if (strstr(line, "MACRO")) {
            inMacro = 1;
            fgets(line, sizeof(line), fp);

            char macroName[20], arg1[20], arg2[20];
            sscanf(line, "%s %s %s", macroName, arg1, arg2);

            strcpy(MNT[mnt_count].name, macroName);
            MNT[mnt_count].mdt_index = mdt_count;
            mnt_count++;

            sprintf(ALA[ala_count].formal, "%s", arg1);
            sprintf(ALA[ala_count].positional, "%d", ala_count + 1);
            ala_count++;

            sprintf(ALA[ala_count].formal, "%s", arg2);
            sprintf(ALA[ala_count].positional, "%d", ala_count + 1);
            ala_count++;

            sprintf(MDT[mdt_count++].definition, "%s &ARG1, &ARG2",
macroName);
        }
        else if (strstr(line, "MEND")) {
            inMacro = 0;
            strcpy(MDT[mdt_count++].definition, "MEND");
        }
        else if (inMacro) {
            replaceArgumentsWithALA(line);
            strcpy(MDT[mdt_count++].definition, line);
        }

        fclose(fp);
    }

// ----- Replace Formal Arguments with ALA Numbers -----
---- void replaceArgumentsWithALA(char *line) {
    for (int i = 0; i < ala_count; i++) {
        char *pos = strstr(line, ALA[i].formal);

```

```

    if (pos) {
        strcpy(pos, ALA[i].positional);
    }
}

// ----- Pass 2: Macro Expansion -----
void processPass2(char *filename) {
    FILE *fp = fopen(filename, "r");
    if (!fp) {
        printf("Error opening file!\n");
        return;
    }

    char line[100];
    int inMacro = 0;

    printf("\nExpanded Source Code:\n");
    while (fgets(line, sizeof(line), fp)) {
        if (strstr(line, "MACRO")) {
            inMacro = 1;
        }
        else if (strstr(line, "MEND")) {
            inMacro = 0;
        }
        else if (!inMacro) {
            int found = 0;

            for (int i = 0; i < mnt_count; i++) {
                if (strstr(line, MNT[i].name)) {
                    found = 1;

                    char macroName[20], actual1[20], actual2[20];
                    sscanf(line, "%s %s %s", macroName, actual1, actual2);

                    char *actualParams[2] = {actual1, actual2};

                    expandMacro(macroName, actualParams);
                    break;
                }
            }

            if (!found) {
                printf("%s", line);
            }
        }
    }

    fclose(fp);
}

```

```

// ----- Macro Expansion with Actual Parameters -----
---- void expandMacro(char *macroName, char *actualParams[])
{   int macroIndex = -1;

    for (int i = 0; i < mnt_count; i++) {
        if (strcmp(MNT[i].name, macroName) == 0) {
            macroIndex = i;
            break;
        }
    }

    if (macroIndex == -1) return;
    int mdtStart = MNT[macroIndex].mdt_index + 1;
    int i = 0;

    while (mdtStart < mdt_count && strcmp(MDT[mdtStart].definition, "MEND")
!= 0) {
        char expandedLine[50];
        strcpy(expandedLine, MDT[mdtStart].definition);

        for (int j = 0; j < 2; j++) {
            char placeholder[5];
            sprintf(placeholder, "#%d", j + 1);
            char *pos = strstr(expandedLine, placeholder);

            if (pos) {
                strncpy(pos, actualParams[j], strlen(actualParams[j]));
                pos[strlen(actualParams[j])] = '\0';
            }
        }

        printf("%s\n", expandedLine);
        mdtStart++;
    }

// ----- Display Tables (MNT, MDT, ALA) -----
void displayTables() {
    printf("\nMacro Name Table (MNT):\n");
    printf("Index Macro Name MDT Index\n");
    for (int i = 0; i < mnt_count; i++) {
        printf("%d %s %d\n", i, MNT[i].name, MNT[i].mdt_index);    }

    printf("\nArgument List Array (ALA):\n");
    for (int i = 0; i < ala_count; i++) {
        printf("%s -> %s\n", ALA[i].formal, ALA[i].positional);    }

    printf("\nMacro Definition Table (MDT):\n");
    printf("Index Definition\n");
    for (int i = 0; i < mdt_count; i++) {

```

```
printf("%d %s\n", i, MDT[i].definition);  
}  
}
```

B.2 Input and Output:

```
macro_input.txt  
1  MACRO  
2  MULTIPLY &ARG1 &ARG2  
3  LOAD &ARG1  
4  MUL &ARG2  
5  STORE RESULT  
6  MEND  
7  
8  MACRO  
9  INCREMENT &ARG  
10 LOAD &ARG  
11 ADD ONE  
12 STORE &ARG  
13 MEND  
14  
15 START  
16 MULTIPLY A, B  
17 INCREMENT X  
18 STOP  
19
```

```
Macro Name Table (MNT):
Index  Macro Name  MDT Index
0      MULTIPLY   0
1      INCREMENT  5

Argument List Array (ALA):
&ARG1 -> #1
&ARG2 -> #2
&ARG  -> #3
&ARG2 -> #4

Macro Definition Table (MDT):
Index  Definition
0      MULTIPLY &ARG1, &ARG2
1      LOAD #1
2      MUL #2
3      STORE RESULT

4      MEND
5      INCREMENT &ARG1, &ARG2
6      LOAD #3
7      ADD ONE

8      STORE #3
9      MEND
```

Expanded Source Code:

```
START
LOAD A,
MUL B
STORE RESULT

LOAD #3
ADD ONE

STORE #3
STOP
```

B.3 Observations and learning:

(Students are expected to comment on the output obtained with clear observations and learning for each task/ sub part assigned)

The **MNT (Macro Name Table)** correctly stores macro names and their starting **MDT index**. The **ALA (Argument List Array)** properly maps formal arguments (&ARG1, &ARG2, &ARG) to positional numbers (#1, #2).

The **MDT (Macro Definition Table)** stores macro definitions, replacing formal arguments with # placeholders.

During **Pass 2 (Expansion)**:

- The macro calls (MULTIPLY A, B and INCREMENT X) are replaced with their definitions.
 - #1 and #2 are correctly substituted with actual parameters.
- The final expanded source code does not contain any macro calls, only expanded assembly instructions.

B.4 Conclusion:

The macro processor correctly identifies, stores, and expands macros.

ALA substitution works as expected, ensuring proper parameter replacement.

The expanded source code is **accurate and ready for further processing** (e.g., assembly).

This experiment demonstrates **how macros simplify repeated instructions**, reducing code redundancy.

B.5 Question of Curiosity

(To be answered by student based on the practical performed and learning/observations)

A. List the features of Macro

Code Reusability – Define once, use multiple times.

Parameterization – Macros accept arguments for flexibility.

Faster Execution – Replaces macro calls with actual code, avoiding function call overhead. Reduced Redundancy – Eliminates repetitive code, making programs concise.

No Memory Overhead – Unlike functions, macros do not require stack operations.

Compile-Time Expansion – Substituted before compilation, improving efficiency.

Better Readability – Simplifies complex code structures.

B. Differentiate Between Macro and Procedure/Function

Feature	Macro	Procedure/Function
Definition	A set of instructions expanded at compile time.	A block of code executed at runtime when called.
Execution	Substituted directly into the code (inline).	Called during execution and uses stack memory.
Speed	Faster, as there is no function call overhead.	Slightly slower due to call and return overhead.
Memory Usage	Can increase code size due to expansion.	Uses less memory as the function is stored once.
Parameter Passing	Uses positional arguments via ALA.	Uses pass-by-value or pass-by-reference.
Recursion	Not possible (since expanded inline).	Possible (can call itself).
Debugging	Difficult due to direct expansion.	Easier since functions have a separate scope.
Scope	Limited to the block where used.	Can be called globally or locally.